

PCReboot v2.0: Проектирование отчужденного ядра на **WinRM** для многоплатформенной автоматизации

Архитектурный фундамент: Трехслойная модель как основа масштабируемости

Разработка программного обеспечения для системной автоматизации требует стратегического подхода, который выходит за рамки простого решения текущей задачи. Цель проекта PCReboot v2.0 — создание не одноразового скрипта, а прочного программного каркаса, способного эволюционировать вместе с меняющимися требованиями [219](#). Фундаментальным решением, обеспечивающим эту долгосрочную жизнеспособность, является принятие трехслойной архитектуры, которая четко разграничивает уровни ответственности: пользовательский интерфейс, бизнес-логика и технологическая реализация. Такой подход напрямую отвечает ключевому требованию пользователя — создать отчуждаемое ядро, которое можно будет переиспользовать и расширять без необходимости полной переработки существующего кода [51](#) [52](#).

Центральное место в этой архитектуре занимает выбор протокола удаленного управления. Анализ показывает явное преимущество WinRM (Windows Remote Management) по сравнению с более традиционными механизмами, такими как удаленный запуск через RPC/SMB (`shutdown /m \\\\.HOST`) [3](#) [4](#). Хотя последний может показаться более простым, его использование несет в себе значительные архитектурные риски. Протокол RPC/SMB является более старым и менее гибким, часто вызывая проблемы с брандмауэрами и политиками безопасности, которые могут блокировать необходимые порты и службы [142](#). Более того, работа через RPC/SMB не тестирует целевой канал управления, что противоречит цели создания универсального каркаса для будущих интеграций [219](#). WinRM, в свою очередь, представляет собой современный, стандартизированный протокол, использующий HTTP/HTTPS, что делает его более совместимым с современными сетевыми инфраструктурами [2](#) [15](#). Он

предоставляет надежный и управляемый канал для выполнения команд, получения результатов и сбора метаданных, что идеально соответствует концепции создания полноценного ядра управления [173](#). Таким образом, решение о выборе WinRM как основного механизма для Windows-хостов является не техническим компромиссом, а осознанной стратегической инвестицией в будущее развитие платформы.

Трехслойная архитектура для PCReboot v2.0 состоит из следующих слоев: 1. **CLI-слой (Интерфейс командной строки)**: Этот слой отвечает исключительно за взаимодействие с пользователем. Он парсит аргументы командной строки (например, список хостов, флаг `dry-run`), предоставляет помощь и форматирует вывод для человека. Ключевая обязанность этого слоя — преобразовать человеческий ввод в структурированные данные, понятные ядру, и затем представить результаты выполнения в удобном для пользователя виде. Использование стандартного модуля `argparse` из библиотеки Python позволяет создать мощный, но при этом легковесный интерфейс без привлечения сторонних зависимостей, что соответствует принципу минимализма [51](#). 2. **Ядро (Core)**: Это мозговой центр утилиты, содержащий всю бизнес-логику перезагрузки. Ядро абстрагировано от конкретных технологий и пользовательских интерфейсов. Оно получает из CLI слоя объект задачи (`RebootTask`), который содержит все необходимые параметры (целевой хост, ID выполнения, флаг `dry-run` и т.д.). Затем ядро выполняет серию шагов: валидацию задачи, подготовку к выполнению, запуск бэкенда и сбор результата. Независимость ядра от CLI и бэкендов достигается за счет использования абстрактных базовых классов и интерфейсов. Именно это разделение позволяет легко заменять или добавлять новые функциональные слои в будущем. 3. **Бэкенд (Backend)**: Этот слой реализует конкретную технологию для взаимодействия с целевым хостом. Для MVP это будет `WinRMBackend`, который использует библиотеку `pywinrm` для отправки команд и получения ответов [185](#). Будущее расширение системы, например, для поддержки Linux-хостов, потребует создания нового бэкенда, такого как `SSHBackend`. Этот новый класс будет реализовывать тот же интерфейс, что и `WinRMBackend`, позволяя ядру взаимодействовать с ним совершенно идентично. Это полностью реализует требование о возможности добавления новых бэкендов без изменения ядра [52](#).

Эта архитектурная модель имеет несколько критически важных преимуществ. Во-первых, она обеспечивает высокую степень модульности. Каждый слой можно разрабатывать, тестировать и поддерживать независимо. Например,

можно улучшить интерфейс командной строки, не затрагивая ни ядро, ни бэкенды. Во-вторых, она повышает отказоустойчивость. Ошибка при работе с одним хостом (например, неверные учетные данные или сетевая недоступность) будет обработана как ошибка конкретной задачи и не приведет к аварийному завершению всей программы. Система сможет продолжить работу с остальными хостами, предоставляя детальный отчет обо всех результатах [52](#). В-третьих, она значительно снижает риски дорогостоящей переделки в будущем. Когда возникнет необходимость интегрировать PCReboot с внешними системами, такими как CMDB или мессенджер BotX, эти интеграции будут реализованы как новые сервисы или плагины, которые потребляют результаты из ядра, а не "вшиваются" в существующий монолит [53](#) [82](#).

Проектирование внутренней модели данных является вторым столпом надежного ядра. Вместо работы с сырыми строками, представляющими имена хостов, система должна оперировать объектами. На вход ядру передается объект задачи, например, **RebootTask**, который инкапсулирует все параметры одной операции перезагрузки: `host`, `run_id`, `dry_run`, `timeout_sec`, `command` и другие [64](#). После выполнения операции для каждого хоста создается объект результата, **RebootResult**. Этот объект должен содержать исчерпывающую информацию о результате: `host`, `status` (например, 'SUCCESS', 'FAILURE'), `message` (текстовое описание), временные метки (`started_at`, `finished_at`), длительность операции, а также исходные параметры (`run_id`, `dry_run`) [82](#) [209](#). Такой подход превращает каждый шаг процесса в хорошо определенную сущность, что упрощает отладку, анализ логов и подготовку данных для будущих интеграций. Вместо того чтобы пытаться извлечь смысл из потока текстовых сообщений, будущие сервисы смогут работать с предсказуемой и структурированной моделью данных [199](#).

В таблице ниже представлено сравнение двух подходов к проектированию, иллюстрирующее преимущества выбранной архитектуры.

Критерий	Одноразовый скрипт (монолит)	Модульная архитектура (PCReboot v2.0)
Структура	Единый файл с непрозрачной логикой.	Четко разделенные слои: CLI, Core, Backend(s).
Расширяемость (новые бэкенды)	Требуется полная перепись кода.	Добавление нового бэкенда сводится к созданию нового класса.
Расширяемость (новые интеграции)	Интеграции "вшиваются" в монолит.	Интеграции являются независимыми сервисами, потребляющими результаты ядра.
Обработка ошибок	Ошибка одного хоста может остановить всю программу.	Ошибка одного хоста не влияет на обработку остальных.
Тестирование	Сложно тестировать отдельные компоненты; сложно воспроизводить сценарии.	Возможность тестировать каждый слой и каждый компонент независимо.
Поддержка	Высокая стоимость поддержки из-за непрозрачности кода.	Низкая стоимость поддержки благодаря модульности и четким границам.
Будущее развитие	Высокий риск "переписывания всего".	Адаптивная архитектура, готовая к новым требованиям.

Таким образом, принятие трехслойной архитектуры с использованием WinRM в качестве основного бэкенда для Windows является не просто техническим выбором, а стратегическим решением, заложившим фундамент для создания надежного, масштабируемого и долговечного программного продукта. Эта модель гарантирует, что первоначальные усилия по проектированию и реализации ядра не будут потрачены впустую, а станут ценным активом для дальнейшего развития платформы PCReboot.

Процесс жизненного цикла операции: от ввода команды до подтверждения перезагрузки

Успешная реализация PCReboot v2.0 зависит не только от правильного выбора архитектуры, но и от детального проектирования каждого этапа жизненного цикла операции перезагрузки. Каждая команда, отправленная пользователем, должна пройти через строго определенную последовательность проверок и действий, чтобы гарантировать безопасность, предсказуемость и полноту информации. Представленный в диалоге процесс, уточненный и дополненный аналитическими данными, формирует основу для надежного ядра системы.

Процесс начинается с инициации, когда оператор предоставляет утилите список целевых хостов. Как было обсуждено ранее, наиболее предпочтительным способом является передача этих данных через аргументы

командной строки, например, с помощью флага `--hosts "PC-01,PC-02"` или `--file hosts.txt` [51](#). Интерактивный ввод через `input()` сокращает количество кода, но он менее удобен для автоматизации и тестирования, поэтому его следует рассматривать как возможное дополнение в будущем [213](#). Получив список, CLI-слой передает его в ядро для дальнейшей обработки.

Следующий, критически важный этап — это валидация и нормализация. Сырой ввод от пользователя всегда может быть грязным: он может содержать лишние пробелы, запятые, дубликаты, имена хостов, которые не резолвятся, или хосты, которые не проходят проверку на соответствие белому списку [66](#) [67](#). Ядро должно выполнить ряд проверок: 1. Нормализация: Разделить строку по запятым, очистить каждое имя хоста от пробелов по краям и привести к единому регистру. 2. Дедупликация: Удалить дублирующиеся записи для избежания лишних операций. 3. Валидация имени: Проверить, что каждое имя хоста соответствует минимальным требованиям (например, содержит точку или достаточно длинное). 4. Проверка на белом списке: Убедиться, что каждый хост находится в заранее согласованном списке разрешенных тестовых машин. Это является ключевой мерой безопасности, предотвращающей случайную перезагрузку серверов в производственной среде [81](#). Если какой-либо хост не проходит валидацию, он помечается со статусом `VALIDATION_FAILED` или `NOT_ALLOWED`, и обработка продолжается для остальных хостов.

После успешной валидации следует этап предварительной проверки. Перед тем как отправлять команду на перезагрузку, необходимо убедиться, что целевой хост доступен и готов к приему команд. Эта проверка состоит из нескольких шагов: 1. Проверка доступности по **ICMP**: Система отправляет несколько ICMP-запросов (ping) для проверки базовой сетевой связности [1](#). Если ping не проходит, это может указывать на серьезные проблемы с сетью или на то, что хост выключен. Однако важно учитывать, что ICMP часто блокируется на файрволах, особенно на доменных контроллерах или в защищенных средах [1](#). Поэтому этот метод не должен быть единственным. 2. Проверка доступности порта **WinRM**: Если проверка по ICMP разрешена и проходит успешно, следует дополнительно проверить доступность порта WinRM (по умолчанию TCP 5985 для HTTP или 5986 для HTTPS) [68](#) [168](#). Это можно сделать с помощью утилиты `telnet` или специализированных PowerShell-команд, таких как `Test-NetConnection` [97](#). Наличие открытого порта WinRM является более надежным признаком того, что служба удаленного управления на целевой машине работает и готова принимать команды. 3. Проверка по **WinRM-идентификации**: Самым надежным способом проверки готовности

WinRM является отправкой специального идентификационного запроса с помощью команды `Test-WSMan <hostname>` в PowerShell [42](#) [43](#). Этот запрос проверяет, что служба WinRM запущена и корректно настроена на прием подключений. Важно отметить, что эта проверка не требует аутентификации, что делает ее идеальной для быстрой диагностики [159](#).

Если любая из предварительных проверок не проходит, хост помечается со статусом `PRECHECK_FAILED`.

Следующий этап — выполнение команды. Если все предыдущие проверки пройдены успешно, ядро готово отправить команду на перезагрузку. Команда `shutdown /r /t <delay> /f` формируется с задержкой, которая может быть настроена как значение по умолчанию (например, 5 секунд) или быть изменена для тестовых целей [79](#). Эта команда передается в выбранный бэкэнд (в данном случае, `WinRMBackend`). Бэкэнд использует библиотеку `pywinrm` для установления соединения с целевым хостом и выполнения команды [2](#). После успешной отправки команды ядро записывает событие `COMMAND_SENT`.

Самый сложный и неточный этап — это подтверждение ухода в перезагрузку. Система должна определить момент, когда хост фактически начал перезагружаться и стал недоступен для управления. Проблема заключается в том, что служба WinRM может продолжать работать даже после того, как операционная система начала процесс перезагрузки, что приводит к ложноположительным результатам при попытке проверить доступность хоста [77](#) [78](#). Аналогичные трудности известны в других системах автоматизации, например, в Ansible модуле `win_reboot` [74](#) [125](#). Для решения этой проблемы PCReboot v2.0 должен использовать эвристическую логику: 1. Ожидание: После отправки команды `shutdown` система ждет определенное время (например, 30-60 секунд), чтобы дать ОС начать процесс перезагрузки. 2. Активная проверка: В течение этого времени система периодически пытается установить соединение с WinRM-портом (TCP 5985/5986). Как только соединение перестает устанавливаться, это является сильным индикатором того, что хост уже выключился. 3. Резервная проверка: Если доступность порта WinRM заблокирована или не может быть проверена, система может вернуться к проверке по ICMP, если она была разрешена на начальном этапе [159](#). 4. Таймаут: Если ни один из методов не дал результата в течение установленного таймаута, задача помечается со статусом `TIMEOUT`. Если система определяет, что хост действительно ушел в перезагрузку, ей нужно еще некоторое время дождаться его полного включения и готовности к приему

команд снова. Этот этап, называемый `post-check`, также требует наблюдения за доступностью сервисов на хосте.

Завершающий этап — формирование отчета и логирования. По окончании каждой операции для каждого хоста создается объект `RebootResult`, который содержит итоговый статус ('REBOOT_CONFIRMED', 'TIMEOUT' или другой код ошибки), подробное сообщение, временные метки и другую полезную информацию [82](#). Эти результаты используются CLI-слоем для формирования человекочитаемого отчета в консоли и записываются в JSONL-файл для машинной обработки и долгосрочного хранения [18](#) [83](#). Поле `run_id` в каждом лог-сообщении обеспечивает возможность коррелировать все действия, связанные с одной сессией запуска утилиты [148](#).

Таким образом, детальное проектирование каждого шага жизненного цикла операции превращает простую задачу перезагрузки в комплексный и надежный процесс управления, который минимизирует риски, обеспечивает полную прозрачность и создает основу для дальнейшего развития.

Технологический стек и реализация ядра: Выбор инструментов для надежности и поддержки

Выбор технологического стека является решающим фактором при разработке долговечного и поддерживаемого программного обеспечения. Для PCReboot v2.0 предпочтение отдается инструментам из стандартной библиотеки Python и проверенным, легковесным сторонним библиотекам, что соответствует принципу минимализма и снижает количество внешних зависимостей, являющихся потенциальным источником уязвимостей и сложностей при развертывании [51](#). Этот подход позволяет сосредоточиться на реализации бизнес-логики, а не на управлении экосистемой зависимостей.

Язык программирования: Python 3.10+ Использование современной версии Python (3.10 или выше) обеспечивает доступ к последним улучшениям языка, таким как продвинутые аннотации типов, которые помогают писать более надежный и самодокументируемый код, а также новые синтаксические конструкции, улучшающие читаемость и поддержку [51](#). Версия 3.10+ также предлагает значительные улучшения в производительности и отладке.

Управление конкурентностью: `concurrent.futures.ThreadPoolExecutor`

Поскольку операция перезагрузки является операцией, ограниченной вводом-выводом (I/O-bound), поскольку она в основном состоит из сетевых взаимодействий (WinRM, ICMP), наиболее эффективным подходом к параллельной обработке является использование многопоточности. Вместо немедленного перехода к сложному фреймворку `asyncio`, который требует глубокого понимания асинхронного программирования, для MVP предлагается использовать высокоуровневый интерфейс

`concurrent.futures.ThreadPoolExecutor` [40](#) [132](#). Этот инструмент из стандартной библиотеки позволяет легко создать пул рабочих потоков и распределить задачи (перезагрузку каждого хоста) между ними.

`ThreadPoolExecutor` берет на себя управление жизненным циклом потоков и сбором их результатов, что значительно упрощает код [132](#). При необходимости для достижения еще большей производительности в будущем можно будет рассмотреть переход на `asyncio`, но для стартового варианта

`ThreadPoolExecutor` является более простым и надежным решением [37](#) [193](#).

Работа с WinRM: `pywinrm` Для взаимодействия с WinRM-сервисом на целевых Windows-хостах используется библиотека `pywinrm` [2](#) [15](#). Она является стандартным, широко используемым и хорошо документированным клиентом WinRM для Python [185](#)[186](#). Библиотека предоставляет простой API для выполнения команд, передачи аргументов и получения выходных данных. Хотя существуют и асинхронные альтернативы, такие как `aio-winrm` [129](#)[130](#), они могут быть менее зрелыми и иметь меньшую поддержку сообщества. Для MVP использование `pywinrm` является наиболее безопасным и быстрым путем к достижению цели.

Логирование: Стандартный модуль `logging` Для логирования событий используется стандартный модуль `logging` Python, который является мощным и гибким инструментом [150](#). Вместо множества разрозненных операторов `print()`, которые трудно контролировать и направлять, система будет использовать централизованный логгер. Конфигурация логгера будет настроена таким образом, чтобы форматировать все сообщения в структурированный JSON, что соответствует требованию ТЗ [18](#) [20](#). Это позволит легко интегрировать логи в системы централизованного сбора и анализа (SIEM, ELK Stack и т.д.) [27](#) [28](#). Использование стандартного модуля также оставляет гибкость для дальнейшего улучшения: при необходимости можно легко переключиться на

более продвинутые решения, такие как `structlog`, не меняя основной код приложения [23](#).

Интерфейс командной строки: **argparse** Вместо использования сторонних фреймворков, таких как `Click` или `Typer`, для разбора аргументов командной строки рекомендуется использовать встроенный модуль `argparse` [51](#). Его функциональности достаточно для MVP, и он не добавляет никаких внешних зависимостей. `argparse` позволяет легко определить обязательные и необязательные флаги (например, `--hosts`, `--file`, `--dry-run`), проверять их значения и предоставлять информативные сообщения об ошибках при неверном использовании утилиты [12](#) [66](#). Это соответствует лучшим практикам проектирования CLI-интерфейсов, которые должны быть предсказуемыми и совместимыми с другими утилитами UNIX-подобных систем [11](#).

Протокол и безопасность **WinRM** При работе с WinRM необходимо учитывать вопросы безопасности. По умолчанию WinRM использует HTTP на порту 5985, что передает данные в открытом виде [45](#). Для более безопасной среды настоятельно рекомендуется настраивать WinRM на использование HTTPS на порту 5986 [70](#) [111](#). Это требует установки SSL-сертификата на целевых серверах и его доверия на машинах-клиентах. `pywinrm` поддерживает подключение по HTTPS и позволяет указывать параметры для проверки сертификата, что помогает защититься от атак типа "человек посередине" [13](#). При подключении по HTTPS клиент будет проверять, что имя хоста в URL совпадает с именем в сертификате (проверка соответствия имени хоста), и что сертификат был выдан доверенным центром сертификации [180](#)[181](#). Некорректная настройка сертификатов является частой причиной ошибок подключения [179](#)[211](#).

Взаимодействие с ядром: Объекты задач и результатов Как было описано ранее, ключевым элементом ядра является использование объектов для представления данных. Это достигается через определение классов Python, таких как `RebootTask` и `RebootResult`. Эти классы могут быть реализованы как простые `dataclasses` или `NamedTuple`, что обеспечивает строгую типизацию и легкость работы с данными.

Примерная структура ядра может выглядеть следующим образом:

```
## core/models.py
from dataclasses import dataclass
from typing import Optional
```

```
@dataclass
class RebootTask:
    host: str
    run_id: str
    delay: int = 5
    dry_run: bool = False
    # ... другие поля
```

```
@dataclass
class RebootResult:
    host: str
    run_id: str
    status: str # e.g., 'SUCCESS', 'FAIL', 'TIMEOUT'
    message: str
    started_at: datetime
    finished_at: datetime
    duration_ms: float
    dry_run: bool
```

Ядро управления: **RebootManager** Основной класс, управляющий процессом, может называться **RebootManager**. Он будет инкапсулировать всю логику: 1. Принимать список **RebootTask**. 2. Последовательно выполнять валидацию для каждой задачи. 3. Для каждой валидной задачи создавать и запускать фоновую задачу (например, с помощью `executor.submit(worker_function, task)`). 4. Собирать и обрабатывать результаты из фоновых потоков. 5. Возвращать итоговый список **RebootResult**.

Этот подход позволяет отделить логику ядра от конкретных реализаций CLI и бэкендов, делая код чистым, модульным и легко тестируемым.

Управление данными и ошибками: Валидация, логирование и отказоустойчивость

Надежная система управления перезагрузкой не может существовать без продуманного механизма обработки данных и ошибок. Проектирование PCReboot v2.0 должно уделять особое внимание трем ключевым областям:

строгая валидация входных данных, структурированное логирование для обеспечения прозрачности и полной отказоустойчивости, гарантирующей, что сбой одного компонента не повлияет на общую работу. Эти элементы являются фундаментом, на котором строится доверие к инструменту.

Валидация и нормализация данных — первый барьер защиты. Как уже упоминалось, сырой ввод от пользователя (например, строка " PC-01 , PC-02 " или список из файла) должен проходить через строгий процесс очистки и проверки [66](#). Этот процесс должен быть реализован на раннем этапе, сразу после получения данных из CLI-слоя, еще до того, как они станут частью бизнес-логики. Ключевые шаги валидации включают:

- **Разбор и очистка:** Разделение строки по запятым, удаление пробелов по краям, приведение к единому регистру (например, нижнему) для унификации.
- **Проверка на пустоту:** Убедиться, что после очистки список хостов не пуст.
- **Форматирование имен:** Проверить, что каждое имя хоста соответствует минимальным шаблонам (например, содержит хотя бы одну точку или имеет достаточную длину).
- **Дедупликация:** Использование множества для автоматического удаления дублирующихся имен.
- **Проверка на белый список:** Реализация механизма, который сравнивает имена хостов с заранее определенным списком разрешенных устройств [81](#). Это критически важная мера безопасности, предотвращающая случайную перезагрузку оборудования вне тестовой среды.
- **Валидация IP-адресов (опционально):** Для более строгой проверки можно добавить валидацию, которая проверяет, резолвится ли имя хоста в валидный IP-адрес [117](#).

Все эти проверки должны быть реализованы с помощью функций, возвращающих четкие результаты. Если валидация проваливается, задача немедленно помечается со статусом **VALIDATION_FAILED** или **NOT_ALLOWED** с соответствующим сообщением об ошибке, и обработка продолжается для следующих хостов.

Структурированное логирование в формате **JSONL** — второй камертон надежности. Вместо того чтобы раскидывать по коду операторы `print()` или использовать примитивные файловые логи, система должна использовать централизованную систему логирования, основанную на стандартном модуле **logging** Python [150](#). Каждое событие в жизненном цикле операции (от начала

до конца) должно записываться как отдельная строка в формате JSONL (JSON Lines) [83](#). Формат JSONL предполагает, что каждая строка в файле является отдельным, валидным JSON-объектом. Это делает логи одновременно и машинно-читаемыми, и человекочитаемыми (при просмотре в правильном редакторе) [22](#).

Каждое лог-сообщение должно содержать богатый набор полей, чтобы обеспечить максимальную информативность. Рекомендуемая схема для лога может включать:

- ``timestamp``: Временная метка события в стандарте ISO 8601.
- ``level``: Уровень логирования (INFO, WARNING, ERROR, DEBUG).
- ``logger``: Имя логгера (например, ``pcr.core``).
- ``run_id``: Уникальный идентификатор сессии запуска, связывающий все действия одной команды.
- ``host``: Имя целевого хоста.
- ``task_id``: Уникальный идентификатор задачи (если есть).
- ``status``: Статус выполнения задачи (например, 'PENDING', 'COMMAND_SENT', 'REBOOT_CONFIRMED').
- ``message``: Описание события или ошибки.
- ``duration_ms``: Время выполнения операции в миллисекундах.
- ``correlation_id``: (опционально) ID для отслеживания запросов в более сложных системах.
- ``extra_fields``: Любые дополнительные данные, например, код ошибки WinRM или результат предварительной проверки.

Такой подход к логированию обеспечивает множество преимуществ. Во-первых, он позволяет легко агрегировать и анализировать логи с помощью стандартных инструментов (Fluent Bit, Logstash, Splunk и т.д.) [83](#). Во-вторых, он обеспечивает полную прозрачность, позволяя точно воспроизвести ход событий в случае сбоя. В-третьих, он готовит почву для будущих интеграций, где логи могут автоматически отправляться в системы управления инцидентами, такие как ServiceNow или LogicMonitor [53](#) [55](#).

Отказоустойчивость и обработка ошибок — третий аспект. Система должна быть спроектирована так, чтобы она могла справиться с неожиданными ситуациями, не прекращая свою работу.

- Обработка ошибок на уровне хоста: Главное правило — ошибка при обработке одного хоста не должна приводить к падению всей программы.

При параллельной обработке с помощью `ThreadPoolExecutor` каждая задача выполняется в отдельном потоке. Исключения, возникающие в потоке, не распространяются на главный поток. Вместо этого они сохраняются в объекте `Future`, который затем можно получить, и обработать, сохранив информацию об ошибке в объекте `RebootResult`. Это позволяет собрать полный отчет обо всех хостах, успешно обработанных и неудачно обработанных.

- **Dry-run** режим: Реализация безопасного режима является ключевой частью отказоустойчивости. Этот режим должен полностью имитировать процесс, но не выполнять никаких деструктивных действий [10 50](#). CLI-слой должен проверять наличие флага `--dry-run`. Если он установлен, ядро должно выполнить все шаги: валидацию, предварительную проверку, и для каждого хоста, прошедшего проверку, сгенерировать и записать в лог сообщение о том, что именно было бы сделано (например, "Планируется отправка команды shutdown на хост PC-01"). Это позволяет оператору увидеть результат своей команды и убедиться в ее безопасности перед фактическим выполнением.
- Управление секретами: Хранение учетных данных (логин, пароль) в коде или конфигурационных файлах является грубым нарушением принципов информационной безопасности [51](#). Для тестовых сред можно использовать файл `.env` с переменными окружения, которые загружаются перед запуском скрипта. В производственной среде необходимо использовать специализированные системы управления секретами, такие как HashiCorp Vault или Azure Key Vault. `Typer` (или аналогичный инструмент) позволяет легко читать значения из переменных окружения [51](#).
- Параллельность и потокобезопасность: При параллельной обработке нескольких хостов запись в общий лог-файл должна быть потокобезопасной. Стандартный модуль `logging` Python с его конфигурацией `dictConfig` или `fileConfig` уже обеспечивает потокобезопасную запись, что делает его подходящим выбором для данной задачи.
- Настройка таймаутов: Для сетевых операций, таких как WinRM, критически важно правильно настраивать таймауты. `pywinrm` позволяет задать таймауты на соединение и чтение. Неправильно выбранные таймауты могут привести к зависанию программы на длительное время при сетевых проблемах или медленном ответе сервера [114216](#). Необходимо установить разумные значения, которые позволят быстро обнаружить

проблемы, но при этом не будут слишком чувствительными к кратковременным задержкам в сети.

Внедрение этих практик управления данными и ошибками превращает PCReboot v2.0 из простого скрипта в профессиональный инструмент, который можно доверять в критически важных операциях. Строгая валидация предотвращает ошибки, структурированное логирование обеспечивает прозрачность, а отказоустойчивость гарантирует надежность.

Перспективы развития: Интеграция внешних систем и добавление новых бэкендов

Главное преимущество архитектурного подхода, принятого для PCReboot v2.0, заключается не только в надежности его текущей реализации, но и в его огромном потенциале для будущего развития. Проектирование системы с четким разделением слоев и использованием абстрактных интерфейсов позволяет легко добавлять новые функциональные возможности, такие как поддержка Linux-систем через SSH и интеграцию с внешними сервисами управления IT-инфраструктурой, не затрагивая и не нарушая существующее ядро. Это обеспечивает долгосрочную ценность инвестиций в разработку и адаптивность платформы к меняющимся требованиям.

Добавление нового бэкенда: Переход на SSH для Linux Перспектива добавления поддержки Linux-хостов является наиболее прямым следствием модульной архитектуры. Процесс добавления нового бэкенда сводится к нескольким четким шагам: 1. Определение интерфейса бэкенда: В ядре должен существовать абстрактный базовый класс (например, **BaseBackend**), который определяет единый контракт для всех бэкендов. Этот класс будет содержать декораторы `@abstractmethod` для методов, которые должны быть реализованы в каждом конкретном бэкенде. Для нашей задачи это могут быть методы: `precheck(self, host: str) -> Tuple[bool, str]`, `reboot(self, host: str, delay: int) -> Tuple[bool, str]`, и возможно, `is_rebooted(self, host: str) -> Tuple[bool, str]`. 2. Создание нового бэкенда: Необходимо создать новый класс, например, **SSHBackend**, который будет наследоваться от **BaseBackend**. Внутри этого класса будет реализована вся логика взаимодействия с целевым хостом через протокол SSH. Для этого можно использовать популярную Python-библиотеку, такую как **paramiko** или **fabric**.

3. Реализация методов: В классе **SSHBackend** необходимо будет реализовать логику для каждой функции из интерфейса. Например, метод **precheck** будет использовать **paramiko** для попытки установить SSH-соединение с хостом. Метод **reboot** будет отправлять команду **shutdown -r +\${delay}**. Метод **is_rebooted** может проверять, отвечает ли хост на SSH-соединение, или выполнять более сложные проверки. 4. Регистрация бэкенда в ядре: Ядро должно иметь механизм для выбора и использования нужного бэкенда. Это может быть реализовано через фабричный метод или через конфигурацию. Например, пользователь может указать флаг **--backend ssh** при запуске утилиты. Ядро, получив этот флаг, создаст экземпляр **SSHBackend** вместо **WinRMBackend**.

Такой подход, основанный на принципах объектно-ориентированного программирования, позволяет легко расширять функциональность системы, не изменяя ее ядро. Это пример хорошего применения паттерна "Стратегия", где различные алгоритмы (WinRM, SSH) могут быть выбраны и заменены во время выполнения.

Интеграция с внешними системами: **CMDB, Naumen, BotX** Аналогичным образом, интеграция с внешними системами, такими как **ServiceNow (CMDB)**, **Naumen** или **BotX**, может быть реализована как независимый модуль или сервис, который работает "параллельно" с основным процессом. Этот подход полностью соответствует требованию "не вшиваться в монолит". 1. Модель данных для интеграции: Ключ к успеху здесь — использование той же структурированной модели данных (**RebootResult**), которую генерирует ядро. Этот объект уже содержит всю необходимую информацию: идентификатор выполнения, имя хоста, статус, сообщение об ошибке, временные метки. Эта модель становится своего рода "стандартным языком" для обмена информацией между ядром и интеграционными модулями. 2. Реализация интеграционного модуля: Для каждой системы-получателя создается свой модуль. Например, **NaumenNotifier**, **BotXSender** или **CMDBSyncer**. Эти модули не являются частью ядра, а представляют собой отдельные скрипты или сервисы. 3. Механизм подписки на события: Ядро может быть спроектировано так, чтобы уведомлять внешние модули о завершении задачи. Это можно реализовать несколькими способами:

- Простой вызов: После завершения обработки всех хостов ядро может запустить внешний скрипт, передав ему путь к файлу с результатами (например, JSONL-лог).

- Паттерн "Наблюдатель": Ядро может поддерживать список "слушателей" (callback-функций). Передаваемый в них объект `RebootResult` позволяет модулям реагировать на события в реальном времени.
- Потребитель очереди: Результаты задач могут публиковаться в очередь сообщений (например, RabbitMQ, Kafka), а интеграционные модули будут выступать в роли потребителей, обрабатывая сообщения из этой очереди.

1. Примеры интеграций:

- **ServiceNow (CMDB):** Интеграционный модуль может получать результат успешной перезагрузки и использовать REST API ServiceNow для создания, обновления или закрытия инцидента, связывая его с конкретным CI (Configuration Item) в CMDB [57](#) [88](#). Логи могут использоваться для enriching (обогащения) данных в CMDB [90](#).
- **BotX/CTS:** Модуль `BotXSender` может получать результат и отправлять пользователю в мессенджере сообщение в формате, соответствующем требованиям API CTS [95](#) [156](#). Это может быть как простое уведомление об успехе, так и детальный отчет об ошибке.
- **Webhooks:** Большинство современных систем (LogicMonitor, PagerDuty, Sumo Logic и др.) поддерживают интеграцию через вебхуки — HTTP POST-запросы, которые система-получатель отправляет при наступлении определенного события [121](#)[202](#). Интеграционный модуль может быть реализован как простой скрипт, который принимает `RebootResult` и формирует JSON-тело для POST-запроса на URL, предоставленный внешней системой [205](#).

Такой плагинообразный подход к интеграциям имеет неоспоримые преимущества. Во-первых, он обеспечивает полную независимость. Можно добавить поддержку ServiceNow, не затрагивая логику работы с WinRM. Во-вторых, он упрощает разработку и тестирование. Интеграционные модули могут разрабатываться и тестироваться независимо от основной платформы. В-третьих, он создает экосистему, где любой участник может создать свой собственный модуль для интеграции с нужной ему системой, используя стандартный формат данных, предоставляемый PCReboot v2.0.

В таблице ниже приведено сравнение двух подходов к интеграции.

Подход	Встроенная интеграция	Плагинообразная интеграция
Структура	Интеграционный код находится внутри ядра.	Интеграционные модули — это отдельные файлы/проекты.
Гибкость	Низкая. Добавление новой интеграции требует изменения и перекомпиляции ядра.	Высокая. Новые интеграции могут быть добавлены без изменения ядра.
Тестирование	Сложное. Требуется тестировать все интеграции вместе с ядром.	Простое. Каждый модуль тестируется независимо.
Поддержка	Высокая стоимость поддержки. Изменение в одном месте может повлиять на все.	Низкая стоимость поддержки. Модули изолированы.
Скорость разработки	Медленная. Каждая новая интеграция — это задача для основной команды.	Быстрая. Сообщество или другие команды могут разрабатывать свои модули.

Таким образом, архитектура PCReboot v2.0 не только решает текущую задачу, но и создает прочный фундамент для будущего роста. Она превращает утилиту из простого инструмента в универсальную платформу для управления перезагрузками, способную бесшовно интегрироваться в любую IT-инфраструктуру.

План внедрения и заключительные рекомендации

Завершающим этапом исследования является формулирование практического плана внедрения разработанного каркаса PCReboot v2.0. Этот план должен быть последовательным, поэтапным и сфокусированным на подтверждении жизнеспособности архитектурных решений на каждом шаге. Он переводит теоретическое проектирование в плоскость практических действий, минимизируя риски и обеспечивая контроль над процессом разработки. Рекомендуемый план включает три ключевых этапа: диагностику среды, итеративную разработку MVP и тестирование.

Этап 0: Диагностика среды (Не является частью кода) Прежде чем написать первую строку кода, абсолютно необходимо убедиться, что целевая среда готова к работе с WinRM. Этот этап является критически важным, так как если WinRM не настроен или не работает, вся выбранная архитектура становится нежизнеспособной.

1. Идентификация тестовой среды: Определить один-два тестовых Windows-хоста (физические машины или виртуальные машины), которые будут использоваться для разработки и тестирования. Убедиться, что машина-инициатор (home-машина) находится в той же доменной среде, если

планируется использование Kerberos-аутентификации [133](#). 2. Проверка наличия и состояния **WinRM**: На целевом Windows-хосте необходимо проверить, что служба WinRM запущена. Это можно сделать через `services.msc` или командой `Get-Service WinRM` в PowerShell [207](#). Также необходимо убедиться, что служба настроена для запуска автоматически. 3. Проверка конфигурации **WinRM**: Запустить на целевом хосте команду `winrm quickconfig` или `Enable-PSRemoting -Force`. Эти команды проверят конфигурацию WinRM и создадут необходимые правила в брандмауэре, если они отсутствуют [44](#) [47](#). Важно отметить, что `winrm quickconfig` может создавать правила только для текущего профиля пользователя, поэтому для серверных сред может потребоваться выполнение команды с повышенными привилегиями [69](#). 4. Ручное тестирование подключения: С машины-инициатора необходимо вручную проверить, что WinRM работает. Это делается с помощью следующих PowerShell-команд:

- ``Test-WSMan <имя_хоста>``: Проверяет, что служба WinRM на удаленном компьютере запущена и доступна [42](#) [97](#).
- ``Test-NetConnection <имя_хоста> -Port 5985``: Проверяет доступность TCP-порта WinRM HTTP [168](#).
- ``Invoke-Command -ComputerName <имя_хоста> -ScriptBlock { hostname }``: Попытка выполнить простую команду для проверки аутентификации и полной работоспособности канала [164](#).

1. Проверка доступности **ICMP**: Убедиться, что ICMP echo request (ping) разрешен и до целевого хоста можно добраться [1](#).

Только после успешного прохождения всех этих шагов можно считать среду готовой к разработке. Если какие-либо шаги провалились, необходимо обратиться к инженерам по инфраструктуре для устранения проблем.

Этап 1: Разработка ядра и CLI (MVP) На этом этапе происходит написание основного кода. 1. Настройка проекта: Создать структуру проекта (`pcr-reboot/`, `pcr_reboot/`, `tests/`, `pyproject.toml`). Настроить `pyproject.toml` для создания исполняемого файла командной строки. 2. Модели данных: Определить классы `RebootTask` и `RebootResult` с помощью `dataclasses`. 3. CLI-слой: Реализовать парсер аргументов с помощью `argparse`. Определить флаги `--hosts`, `--file` и `--dry-run`. Написать функцию, которая читает хосты из аргументов или файла и преобразует их в список объектов `RebootTask`. 4. Модуль `core/__init__.py`: Создать

основной класс **RebootManager**, который будет принимать список **RebootTask** и возвращать список **RebootResult**. 5. Модуль **backends/winrm_backend.py**: Реализовать класс **WinRMBackend**, который будет иметь методы **precheck**, **reboot** и **is_rebooted**. Использовать библиотеку **pywinrm** для выполнения операций. Реализовать логику аутентификации (например, по доменному сервисному аккаунту). 6. Основная логика в **RebootManager**: В **RebootManager** реализовать последовательность вызовов: для каждой задачи выполнить предварительную проверку через **WinRMBackend**, если она прошла, выполнить команду перезагрузки и подтвердить ее завершение. 7. Логирование: Настроить централизованное логирование с выводом в JSONL-формат в отдельный файл, имя которого может генерироваться на основе **run_id** или временной метки.

Этап 2: Тестирование После написания кода необходимо провести его тщательное тестирование. 1. Юнит-тесты: Написать юнит-тесты для каждой функции и метода, особенно для логики валидации, обработки ошибок и модели данных. 2. Тесты на интеграцию: Провести ручные тесты с использованием реальной тестовой среды. Прогнать сценарии:

- Успешная перезагрузка: Перезагрузка одного или нескольких доступных хостов.
- **Dry-run**: Проверка, что в режиме `--dry-run` ничего не перезагружается, а все действия логируются.
- Ошибка валидации: Попытка перезагрузить хост, не входящий в белый список.
- Сетевая ошибка: Попытка перезагрузить недоступный хост (например, с выключенным Wi-Fi).
- Ошибка аутентификации: Попытка запустить скрипт с неверными учетными данными.
- Параллельная обработка: Перезагрузка нескольких хостов одновременно, чтобы убедиться, что они обрабатываются параллельно, а ошибки не влияют на друг друга.
- Таймаут: Проверка поведения системы при истечении таймута ожидания.

Заключительные рекомендации Разработка PCReboot v2.0 представляет собой не просто автоматизацию одной команды, а создание программного каркаса для управления перезагрузками. Предложенная архитектура с трехслойной моделью (CLI-Core-Backend), основанная на модульности и абстракции, является наиболее надежным и масштабируемым решением. Выбор WinRM как основного транспорта для Windows-хостов соответствует современным стандартам и

заложен фундамент для будущего добавления новых бэкендов (например, SSH для Linux) и интеграций с внешними системами (CMDB, BotX) без необходимости переписывания ядра.

Ключ к успеху проекта лежит в следующих рекомендациях:

- Сделайте упор на ядро: Ваша первоочередная цель — не просто сделать скрипт, который работает, а создать хорошо спроектированное, тестируемое и документированное ядро, которое инкапсулирует всю логику.
- Не пренебрегайте диагностикой: Перед тем как начать писать код, проведите тщательную ручную диагностику вашей тестовой среды. Убедитесь, что WinRM, порты и сетевая связность работают корректно. Это сэкономит много времени в будущем.
- Следуйте принципу "разделяй и властвуй": Разделяйте CLI, Core и Backend на разные модули. Не позволяйте слоям друг на друга "смотреть". Ядро должно знать только об абстрактном бэкенде, а не о конкретной реализации WinRM.
- Начните с простого, но надежного: Используйте ``concurrent.futures.ThreadPoolExecutor`` вместо ``asyncio`` для MVP. Это упростит разработку и отладку. ``asyncio`` можно будет рассмотреть позже, если будет необходимость в очень высокой степени параллелизма.
- Помните о будущем: Каждое решение, принятое сегодня, должно продумываться с точки зрения его влияния на будущие расширения. Используйте абстрактные классы, структурированные данные (JSON) и четкие интерфейсы. Это окупится в будущем, когда потребуется добавить поддержку Linux или интеграцию с ServiceNow.

Следуя этому исследовательскому отчету и плану внедрения, вы сможете разработать не просто рабочий инструмент, а прочный и долговечный программный продукт, который станет ценным активом для вашего отдела.

Справка

1. Use Ping to Troubleshoot Your Network - Radio World <https://www.radioworld.com/news-and-business/use-ping-to-troubleshoot-your-network>

2. diyan/pywinrm: Python library for Windows Remote ... <https://github.com/diyan/pywinrm>
3. Enabling Remote Shutdown in Active Directory | by Root <https://medium.com/@basharraed/enabling-remote-shutdown-in-active-directory-cbf7ba435e4b>
4. how to shut down pc with "net rpc shutdown" <https://learn.microsoft.com/en-us/answers/questions/5488706/how-to-shut-down-pc-with-net-rpc-shutdown>
5. Orion - Windows Service Monitoring - WMI Vs. RPC <https://thwack.solarwinds.com/discussion/48244/orion-windows-service-monitoring-wmi-vs-rpc>
6. Windows 11 net rpc shutdown <https://superuser.com/questions/1746456/windows-11-net-rpc-shutdown>
7. Getting error in connecting with remote Windows server using WinRM <https://learn.microsoft.com/en-us/answers/questions/2190196/getting-error-in-connecting-with-remote-windows-se>
8. Troubleshoot a WinRM Connection · digital.ai Release <https://docs.digital.ai/release/docs/xl-platform/how-to/troubleshoot-a-winrm-connection>
9. Solved: How do I repair WinRM? - Experts Exchange <https://www.experts-exchange.com/questions/24997695/How-do-I-repair-WinRM.html>
10. In praise of --dry-run | Hacker News <https://news.ycombinator.com/item?id=27263136>
11. What are good habits for designing command line arguments? <https://softwareengineering.stackexchange.com/questions/307467/what-are-good-habits-for-designing-command-line-arguments>
12. OR function with argparse with two variables on the command line in ... <https://stackoverflow.com/questions/12536217/or-function-with-argparse-with-two-variables-on-the-command-line-in-python>
13. Windows Remote Management - Ansible Documentation https://cn-ansible.readthedocs.io/zh_CN/latest/user_guide/windows_winrm.html
14. Service overview and network port requirements for Windows <https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/service-overview-and-network-port-requirements>
15. pywinrm <https://pypi.org/project/pywinrm/0.0.2dev/>
16. asyncio — Asynchronous I/O <https://docs.python.org/3/library/asyncio.html>
17. pywinrm - conda-forge <https://anaconda.org/conda-forge/pywinrm>
18. Python Logging Module: A Complete Guide <https://www.dash0.com/guides/logging-in-python>
19. Structured Logging Best Practices for Production in 2026 <https://www.grepr.ai/blog/structured-logging-best-practices>

20. 5 best logging libraries for Python <https://launchdarkly.com/blog/why-use-logging-libraries-for-python/>
21. Guide to structured logging in Python <https://newrelic.com/blog/log/python-structured-logging>
22. Structured logs are great... until you actually have to read ... https://www.reddit.com/r/golang/comments/1rfile6/structured_logs_are_great_until_you_actually_have/
23. Logging Best Practices <https://www.structlog.org/en/stable/logging-best-practices.html>
24. JSON Logging Best Practices - Loggly <https://www.loggly.com/use-cases/json-logging-best-practices/>
25. Python best practice in terms of logging <https://stackoverflow.com/questions/22807972/python-best-practice-in-terms-of-logging>
26. Logging Cookbook — Python 3.14.7 documentation <https://docs.python.org/3/howto/logging-cookbook.html>
27. Logs Ingestion API in Azure Monitor - Microsoft Learn <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/logs-ingestion-api-overview>
28. Log ingestion API - Dynatrace Documentation <https://docs.dynatrace.com/docs/analyze-explore-automate/logs/lma-log-ingestion/lma-log-ingestion-via-api>
29. Supported Logs and Ingestion Methods - LogicMonitor <https://www.logicmonitor.com/support/supported-logs-and-ingestion-methods>
30. Logs Ingestion API - Illumio <https://product-docs-repo.illumio.com/Tech-Docs/Integrations/out/en/illumio-sentinel-solution-3-4-1/deploy-the-illumio-sentinel-solution/logs-ingestion-api.html>
31. Azure Logs Ingestion API | 4.0 | Fluent Bit: Official Manual https://docs.fluentbit.io/manual/4.0/data-pipeline/outputs/azure_logs_ingestion
32. Ingest logs and data from Box - Administrator Guide - Cortex XSIAM <https://docs-cortex.paloaltonetworks.com/r/Cortex-XSIAM/Cortex-XSIAM-3.x-Documentation/Ingest-logs-and-data-from-Box>
33. Authenticate Log Ingestion - Broadcom TechDocs <https://techdocs.broadcom.com/us/en/ca-enterprise-software/it-operations-management/digital-operational-intelligence/24-1/authenticate-log-ingestion.html>
34. Simplified log ingestion | Elastic Docs <https://www.elastic.co/docs/reference/fleet/elastic-agent-simplified-input-configuration>
35. Chronicle API ingestion methods | Google Security Operations <https://docs.cloud.google.com/chronicle/docs/reference/ingestion-methods>

36. Retrieve Logs Using the Management API - Auth0 Docs <https://auth0.com/docs/deploy-monitor/logs/retrieve-log-events-using-mgmt-api>
37. What's the Best Way to Handle Concurrency in Python - Medium <https://medium.com/towardsdev/whats-the-best-way-to-handle-concurrency-in-python-threadpoolexecutor-or-asyncio-85da1be58557>
38. Parallelism, Concurrency, and AsyncIO in Python - by example <https://testdriven.io/blog/python-concurrency-parallelism/>
39. Why Should Async Get All The Love?: Advanced Control Flow With Threads <https://emptysqua.re/blog/why-should-async-get-all-the-love/>
40. Python ThreadPoolExecutor: Use Cases for Parallel Processing <https://blog.wahab2.com/python-threadpoolexecutor-use-cases-for-parallel-processing-3d5c90fd5634>
41. Python's threadpoolexecutor is still single OS-threaded. There will only ... <https://news.ycombinator.com/item?id=23293096>
42. Test-WSMan - PowerShell - Microsoft Learn <https://learn.microsoft.com/en-us/powershell/module/microsoft.wsman.management/test-wsman?view=powershell-7.6>
43. Running commands on a a computer remotely - Spiceworks Community <https://community.spiceworks.com/t/running-commands-on-a-a-computer-remotely/791772>
44. Configure Server Monitoring Using WinRM - Palo Alto Networks <https://docs.paloaltonetworks.com/pan-os/10-1/pan-os-admin/user-id/map-ip-addresses-to-users/configure-server-monitoring-using-winrm>
45. How to Configure WinRM Over HTTPS on Windows Server 2022/2025 <https://www.youtube.com/watch?v=LZpdTPJLXHo>
46. start winrm remotely when broken : r/PowerShell - Reddit https://www.reddit.com/r/PowerShell/comments/10p4hvr/start_winrm_remotely_when_broken/
47. Allowing WinRM in the Windows Firewall - powershell - Stack Overflow <https://stackoverflow.com/questions/42384177/allowing-winrm-in-the-windows-firewall>
48. Active Directory - Configure Windows Remote Management (WinRM) <https://kb.wisc.edu/iam/35144>
49. The Ultimate Guide to Enabling and Using WinRM - Atera <https://www.atera.com/blog/enable-winrm/>
50. In praise of --dry-run : r/programming https://www.reddit.com/r/programming/comments/nk2t3z/in_praise_of_dryrun/
51. 7 Typer CLI Patterns That Feel Like Real Tools <https://medium.com/@connect.hashblock/7-typer-cli-patterns-that-feel-like-real-tools-ecbe72720828>
52. Command line interface | Snakemake 9.25.1 documentation <https://snakemake.readthedocs.io/en/stable/executing/cli.html>

53. CMDB Integrations Dashboard <https://www.servicenow.com/docs/r/xanadu/servicenow-platform/cmdb-integration-commons/cmdb-integ-dashboard.html>
54. Creating a CMDB integration tutorial using IH-ETL : Part 3 of 5 https://www.youtube.com/watch?v=_4fVx5_tsiY
55. LogicMonitor ServiceNow CMDB Integration Overview <https://www.logicmonitor.com/support/logicmonitor-servicenow-cmdb-integration-overview>
56. ServiceNow CLI for AI Agents <https://composio.dev/toolkits/servicenow/framework/cli>
57. ServiceNow Incident Management Integration with Zero ... <https://manuals.supernaeyeglass.com/project-solutions/HTML/servicenow-incident-management-integration-with-zero-trust-alerts.html>
58. ServiceNow CMDB - Cortex XSIAM - Security Operations <https://docs-cortex.paloaltonetworks.com/r/Cortex-XSIAM/Cortex-XSIAM-3.x-Documentation/ServiceNow-CMDB>
59. ServiceNow | Elastic integrations <https://www.elastic.co/docs/reference/integrations/servicenow>
60. CMDB Integration - OpsRamp Documentation https://_.docs.opsramp.com/integrations/service-management/servicenow/servicenow-using-servicenow-store/cmdb-integration/
61. Webhooks — DataGerry latest documentation <https://docs.datagerry.com/en/latest/usage/webhooks.html>
62. adityatelange/evil-winrm-py <https://github.com/adityatelange/evil-winrm-py>
63. Feedback for evil-winrm-py - python-based tool for ... https://www.reddit.com/r/hackthebox/comments/1majl3q/feedback_for_evilwinrm_py_pythonbased_tool_for/
64. Structured CLI Output: A Best Practice for DevOps Teams - Medium <https://medium.com/metasintaxis/structured-cli-output-a-best-practice-for-devops-teams-5d0d6c1d71f5>
65. Command Line Interface Guidelines <https://clig.dev/>
66. Python: Argument Parsing Validation Best Practices <https://stackoverflow.com/questions/37471636/python-argument-parsing-validation-best-practices>
67. Python argparse Validation — Types, Choices & Safe Input <https://novaxis.ghost.io/lesson-3-validation-errors-data-safety-type-checking/>
68. Problems with Windows Remote Management (WinRM) : r/sysadmin https://www.reddit.com/r/sysadmin/comments/qn8je9/problems_with_windows_remote_management_winrm/

69. Send remote commands with Powershell - Can't find the right port ... <https://serverfault.com/questions/1059075/send-remote-commands-with-powershell-cant-find-the-right-port-to-open>
70. Port 5985/5986 – WinRM (Windows Remote Management) | PentestPad <https://www.pentestpad.com/port-exploit/port-59855986-winrm-windows-remote-management>
71. WinRM - Port 5985, 5986 - VeryLazyTech <https://www.verylazytech.com/winrm-port-5985-5986>
72. ansible.windows.win_reboot module – Reboot a windows machine https://docs.ansible.com/projects/ansible/latest/collections/ansible/windows/win_reboot_module.html
73. How to Use Ansible win_reboot Module - OneUptime <https://oneuptime.com/blog/post/2026-02-21-how-to-use-ansible-win-reboot-module/view>
74. KB5004244 appears to have broken win_reboot on Server 2019 #249 <https://github.com/ansible-collections/ansible.windows/issues/249>
75. Ansible win_reboot — Reboot Windows | AnsibleByExample <https://www.ansiblebyexample.com/articles/reboot-windows-hosts-ansible-module-winreboot>
76. How to automate system reboots using the Ansible reboot module <https://www.redhat.com/en/blog/automate-reboot-ansible>
77. Sick of Wondering if a Remote Computer is done ... https://www.reddit.com/r/PowerShell/comments/8va2fh/sick_of_wondering_if_a_remote_computer_is_done/
78. windows-restart fails to detect complete restart during ... <https://github.com/hashicorp/packer/issues/6799>
79. PowerShell Remote System Management Tutorial <https://www.youtube.com/watch?v=PRlvzWtSz4w>
80. Using the CLI <https://docs.pyinfra.com/en/3.x/cli.html>
81. Python: Validating Argparse Input with a List - Brian T. Carr <https://www.briancarr.org/post/python-validating-argparse-input>
82. cli: exit-code map + structured JSON error envelopes #1564 - GitHub <https://github.com/modelcontextprotocol/inspector/issues/1564>
83. JSONL for Logs - Structured Logging <https://jsonl.help/use-cases/log-processing/>
84. Features Removed or No Longer Developed in Windows ... <https://learn.microsoft.com/en-us/windows-server/get-started/removed-deprecated-features-windows-server>
85. Troubleshoot WMI connectivity and access issues <https://learn.microsoft.com/en-us/troubleshoot/windows-server/system-management-components/scenario-guide-troubleshoot-wmi-connectivity-access-issues>

86. Windows Discovery fails even when credential is Valid <https://www.servicenow.com/community/itom-forum/windows-discovery-fails-even-when-credential-is-valid/td-p/3213699>
87. Configure Kerberos for WMI/WinRM authentication in the ... https://documentation.solarwinds.com/en/success_center/orionplatform/content/core-configure-kerberos.htm
88. ServiceNow (Incident Management) Integration - LogicMonitor <https://www.logicmonitor.com/support/servicenow-incident-management-integration>
89. Integrate Logicmonitor events - ServiceNow <https://www.servicenow.com/docs/r/it-operations-management/event-management/logicmonitor-events-integration.html>
90. LogicMonitor CMDB Integration - ServiceNow Store <https://store.servicenow.com/store/app/6fd9ef621b246a50a85b16db234bcbe5>
91. ServiceNow CMDB Integration. Let's get started. - YouTube https://www.youtube.com/watch?v=Qij_Ju3TmpM
92. LogicMonitor – ServiceNow CMDB & ITSM Integration <https://www.logicmonitor.com/integrations/servicenow-cmdb>
93. LogicMonitor + ServiceNow integration - Tray.ai <https://tray.ai/connectors/logic-monitor-servicenow-integrations/>
94. ServiceNow CMDB Integration - LogicMonitor <https://www.logicmonitor.com/partners/servicenow>
95. Set Up ServiceNow Connections | Sumo Logic Docs <https://www.sumologic.com/help/docs/alerts/webhook-connections/servicenow/set-up-connections/>
96. ServiceNow CMDB Integration. Let's get started. - OpsMatters <https://opsmatters.com/videos/introducing-logicmonitor-servicenow-cmdb-integration-lets-get-started>
97. How to enable and test PowerShell remoting on ... https://support.servicenow.com/kb?id=kb_article_view&sysparm_article=KB0813330
98. Sending Logs to Ingestion API - LogicMonitor <https://www.logicmonitor.com/support/lm-logs/sending-logs-to-ingestion-api>
99. Ingestion APIs - Aisera Product Documentation <https://docs.aisera.com/apis/apis/ingestion-apis>
100. Ingestion metrics table | Google Security Operations <https://docs.cloud.google.com/chronicle/docs/reference/ingestion-metrics-schema>
101. MaintainX – REST API v1 <https://maintainx.dev/>
102. Webhooks service API - Network Node Manager <https://docs.microfocus.com/doc/269/24.3/webhooksapi>

103. ServiceDesk Plus | Documentation and Guides | Apono <https://docs.apono.io/docs/webhook-integrations/request-webhook/itsm/servicedesk-plus>
104. REST API Webhooks - SymphonyAI Summit <https://help.symphonysummitai.com/sauto/docs/rest-api-webhooks>
105. API Ingestion and Webhook Listener | Data Pipeline <https://docs.bdb.ai/data-pipeline-2/components/consumers/api-ingestion-and-webhook-listener>
106. CMMS API and Webhooks for Facility Teams - Infodeck <https://www.infodeck.io/platform/api/>
107. Working with Webhooks - Ivanti https://help.ivanti.com/ht/help/en_US/ISM/2024/admin-user/Content/Configure/Webhooks/Working-with-Webhooks.htm?Highlight=webhook
108. Active Directory Monitoring: The Silent Failure Nobody ... https://www.linkedin.com/posts/girish-m-vyas_ad-monitoring-activity-7444231413543055361-61Wc
109. evil-winrm-py - PyPI <https://pypi.org/project/evil-winrm-py/>
110. evil-winrm-py | Kali Linux Tools <https://www.kali.org/tools/evil-winrm-py/>
111. How to configure WINRM for HTTPS - Windows Client <https://learn.microsoft.com/en-us/troubleshoot/windows-client/system-management-components/configure-winrm-for-https>
112. Structured Logging in Production: Best Practices for ... <https://openobserve.ai/blog/structured-logging-best-practices/>
113. Troubleshooting WinRM errors - PitStop ManageEngine <https://pitstop.manageengine.com/portal/en/kb/articles/troubleshooting-winrm-errors>
114. Timeout handling while using run_in_executor and asyncio <https://stackoverflow.com/questions/34452590/timeout-handling-while-using-run-in-executor-and-asyncio>
115. Guide to Concurrency in Python with Asyncio <https://news.ycombinator.com/item?id=23289563>
116. WinRM is not working - Server Fault <https://serverfault.com/questions/1169533/winrm-is-not-working>
117. Checking for Conditions in Argparse, and Unit Test : r/learnpython - Reddit https://www.reddit.com/r/learnpython/comments/k6qoe7/checking_for_conditions_in_argparse_and_unit_test/
118. Certificate Hostname Verification <https://www.ibm.com/docs/en/tivoli-netcoolimpact/7.1.0?topic=security-certificate-hostname-verification>
119. cli_error.rs - source - Docs.rs https://docs.rs/sublime_cli_tools/latest/src/sublime_cli_tools/error/cli_error.rs.html

120. Re: LogicMonitor Incident Management Integration issues <https://www.servicenow.com/community/sysadmin-forum/logicmonitor-incident-management-integration-issues/m-p/2843603>
121. Custom HTTP Delivery <https://www.logicmonitor.com/support/custom-http-delivery>
122. Configure ServiceNow Webhook Connector - alert logic docs <https://legacy.docs.alertlogic.com/product-guides/console/connectors/servicenow.htm>
123. How to Connect LogicMonitor to ServiceNow Event ... <https://www.servicenow.com/community/itom-blog/how-to-connect-logicmonitor-to-servicenow-event-management-using/ba-p/3541288>
124. Integrating with ServiceNow using Webhooks - Documentation <https://documentation.red-gate.com/monitor14/integrating-with-servicenow-using-webhooks-239668090.html>
125. Ansible 2.8.5 win_reboot never successfully validate ... <https://github.com/ansible/ansible/issues/62343>
126. ansible.builtin.reboot module – Reboot a machine https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/reboot_module.html
127. Logging Best Practices: 12 Tips for Developers and SREs <https://www.parseable.com/blog/logging-best-practices>
128. Structured Logging: Best Practices & JSON Examples <https://uptrace.dev/glossary/structured-logging>
129. Alternatives to pywinrm? · Issue #243 <https://github.com/diyan/pywinrm/issues/243>
130. cournape/aio-winrm: A python 3.5+ async library for the ... <https://github.com/cournape/aio-winrm>
131. Hackman238/txwinrm: Asynchronous Python WinRM client <https://github.com/Hackman238/txwinrm>
132. Concurrency in Python - Threads, Processes, and AsyncIO ... <https://www.thegeekyway.com/concurrency-in-python/>
133. Pywinrm and Active Directory PowerShell cmdlets <https://stackoverflow.com/questions/45306504/pywinrm-and-active-directory-powershell-cmdlets>
134. test access to a remote computer <https://forums.powershell.org/t/test-access-to-a-remote-computer/8871>
135. Error the RPC server is unavailable - Windows <https://learn.microsoft.com/en-us/troubleshoot/windows-server/user-profiles-and-logon/not-log-on-error-rpc-server-unavailable>
136. IPAM Windows DHCP server scan fails on WinRM-only node ... <https://solarwindscore.my.site.com/SuccessCenter/s/article/IPAM-Windows-DHCP-server->

scan-fails-on-WinRM-only-node-with-The-remote-procedure-call-failed-and-did-not-execute-while-PowerShell-WinRM-works

137. RPC server unavailable (WMI troubleshooting) <https://support.vscope.net/troubleshooting/wmi-troubleshoot-rpc-server-is-unavailable/>
138. New-M365DSCReportFromConfiguration: Connection to ... <https://github.com/Microsoft365DSC/Microsoft365DSC/issues/6617>
139. Troubleshooting WMI <https://www.logicmonitor.com/support/monitoring/os-virtualization/troubleshooting-wmi>
140. Confirm WMI access through DCOM/WinRM <https://support.vscope.net/troubleshooting/confirm-wmi-access-through-dcom-winrm/>
141. Server 2022 WMI Issues <https://techcommunity.microsoft.com/discussions/windowsserver/server-2022-wmi-issues/4403952>
142. How to restart a remote computer using Windows <https://www.atera.com/blog/how-to-restart-a-remote-computer-using-windows/>
143. Windows Server install updates playbook inconsistent - Get Help <https://forum.ansible.com/t/windows-server-install-updates-playbook-inconsistent/11024>
144. Reboot Windows hosts — Ansible module win_reboot <https://medium.com/p/f70f271e7a97>
145. ServiceNow Troubleshooting Guide - PagerDuty Knowledge Base <https://support.pagerduty.com/main/docs/servicenow-troubleshooting-guide>
146. Integration with ServiceNow - Dotcom-Monitor <https://www.dotcom-monitor.com/wiki/knowledge-base/integration-with-servicenow/>
147. Service Now Event Integration using Webhook / Rest API Action <https://pitstop.manageengine.com/portal/en/kb/articles/service-now-event-integration-using-webhook-rest-api-action>
148. Structured JSON logging with correlation IDs · Issue #300 · ... <https://github.com/IBM/mcp-context-forge/issues/300>
149. HOW TO IMPLEMENT TODAY: 1. Create an ... <https://www.threads.com/@therobertta/post/DabqTW4mr7Z/how-to-implement-today-create-an-audit-logger-that-writes-jsonl-files-add-one/>
150. logging — Logging facility for Python <https://docs.python.org/3/library/logging.html>
151. Ingest data with the Ingestion API <https://docs.cloud.google.com/chronicle/docs/reference/ingestion-api>
152. Webhook Events as Logs <https://www.logicmonitor.com/support/webhook-events-as-logs>
153. Send data to Azure Monitor using Logs ingestion API ... <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/tutorial-logs-ingestion-api>

154. REST API and Webhooks <https://support.monnit.com/category/360-rest-api-and-webhooks>
155. Does OBM have the capability to receive events via REST ... <https://community.opentext.com/it-ops-cloud/ai-ops-mgmt/w/tips/45415/knowledge-doc-does-obm-have-the-capability-to-receive-events-via-rest-json-webhooks>
156. Logging activity with a webhook <https://www.ibm.com/docs/en/watsonx/watson-orchestrate/base?topic=developing-logging-activity-webhook>
157. Monnit Sensor Integration (Webhook Setup) - Support Center <https://help.gofmx.com/hc/en-us/articles/11791338626573-Monnit-Sensor-Integration-Webhook-Setup>
158. Publish Events to Webhook | MinIO AIStor Documentation <https://docs.min.io/aistor/administration/bucket-notifications/publish-events-to-webhook/>
159. WinRM | Cloud Robots <https://cloudrobots.net/tag/winrm/>
160. Windows Remote Management <https://murrayjm.github.io/WinRM-listeners/>
161. Daily Challenge - Enabling WinRM - 2025-Apr-08 <https://community.spiceworks.com/t/daily-challenge-enabling-winrm-2025-apr-08/1194175>
162. Enhancing WinRM Security: Best Practices for Windows ... <https://wafatech.sa/blog/windows-server/windows-security/enhancing-winrm-security-best-practices-for-windows-server-administration/>
163. Complete Guide: How to Enable WinRM on Windows 10/11 <https://www.ninjaone.com/blog/enable-winrm-on-windows/>
164. Python Pywinrm using powershell to control the AD of ... <https://stackoverflow.com/questions/74134249/python-pywinrm-using-powershell-to-control-the-ad-of-remote-windows-returns-dif>
165. winrmcp <https://pypi.org/project/winrmcp/>
166. Windows security baseline for Windows Server 2025 <https://learn.microsoft.com/en-us/azure/governance/policy/samples/guest-configuration-baseline-windows-server-2025>
167. pywinrm <https://pypi.org/project/pywinrm/>
168. Having trouble with WinRM : r/PowerShell https://www.reddit.com/r/PowerShell/comments/2v10fi/having_trouble_with_winrm/
169. Integrate Logicmonitor events <https://www.servicenow.com/docs/r/zurich/it-operations-management/event-management/logicmonitor-events-integration.html>
170. ServiceNow (Incident Management) Integration <https://www.logicmonitor.com/support/servicenow-integration>
171. Structured Logging Guide and Best Practices - Dash0 <https://www.dash0.com/guides/structured-logging-for-modern-applications>

172. How to Implement Structured Logging Best Practices - OneUptime <https://oneuptime.com/blog/post/2026-01-25-structured-logging-best-practices/view>
173. Package python3-module-winrm: Information <https://packages.altlinux.org/en/p10/srpms/python3-module-winrm/>
174. Logging activity with a webhook <https://www.ibm.com/docs/en/watsonx/watson-orchestrate/base?topic>
175. Webhook Server Logs <https://docs.min.io/aistor/reference/aistor-server/settings/metrics-and-logging/server-logs/>
176. Webhook Tools - Anam <https://anam.ai/docs/personas/tools/webhook-tools>
177. Remote Management of Hyper-V (2022) - Failing <https://community.spiceworks.com/t/remote-management-of-hyper-v-2022-failing/1166065>
178. How to Enable and Configure WinRM for Remote ... <https://www.progressiverobot.com/2025/06/01/how-to-enable-and-configure-winrm-for-remote-management-on-windows-server-2025/>
179. Pywinrm and HTTPS Connection - python - Stack Overflow <https://stackoverflow.com/questions/60567890/pywinrm-and-https-connection>
180. How to verify hostname of certificate? and Is it mandatory if client knows ... <https://security.stackexchange.com/questions/276525/how-to-verify-hostname-of-certificate-and-is-it-mandatory-if-client-knows-the-c>
181. Windows Client Configuration - Secure WinRM <https://jstuyts.github.io/Secure-WinRM-Manual/windows-client-configuration.html>
182. How to Set Up WinRM Over HTTPS with Puppet <https://www.puppet.com/blog/winrm-certificate>
183. Add machine-readable error codes to JSON commands <https://github.com/max-sixty/worktrunk/issues/3697>
184. Teamwork Graph CLI changelog <https://developer.atlassian.com/cloud/twg-cli/changelog/>
185. Releases · diyan/pywinrm <https://github.com/diyan/pywinrm/releases>
186. dev-python/pywinrm <https://summer.exherbo.org/packages/dev-python/pywinrm/index.html>
187. LogicMonitor Incident Management Integration issue... <https://www.servicenow.com/community/sysadmin-forum/logicmonitor-incident-management-integration-issues/m-p/2840480>
188. Creating an incident in ServiceNow https://docs.nexthink.com/platform/configuring_nexthink/bringing-data-into-your-nexthink-instance/integrating-nexthink-with-third-party-tools/outbound-connectors/webhooks/webhook-use-cases-setup/creating-an-incident-in-servicenow

189. WinRS (WinRM) fails - Ports 135, 5985, and 5986 are filtered <https://community.spiceworks.com/t/winrs-winrm-fails-ports-135-5985-and-5986-are-filtered/685922>
190. Configure the WinRM service for remote Windows monitoring <https://docs.microfocus.com/doc/426/24.4/winrmservicewinmonitoring>
191. Multithreading, Concurrent Futures, and Asyncio for HTTP ... <https://medium.com/@zekkelar1337/comparing-speed-multithreading-concurrent-futures-and-asyncio-for-http-requests-82a2b86e4863>
192. A Deep Dive into Asyncio, Threads, and Multiprocessing https://medium.com/@sohail_saifi/optimizing-python-for-concurrency-a-deep-dive-into-asyncio-threads-and-multiprocessing-2bbde8459304
193. Making Python Faster — ThreadPoolExecutor vs AsyncIO vs ... <https://zlliu.medium.com/making-python-faster-threadpoolexecutor-vs-asyncio-vs-multiprocessing-processpoolexecutor-7b2b10ce1364>
194. JSONL Tools - CLI, Parsers & Validators <https://jsonl.help/tools/>
195. How to Use the Ansible reboot Module Safely - OneUptime <https://oneuptime.com/blog/post/2026-02-21-how-to-use-the-ansible-reboot-module-safely/view>
196. Restart-Computer (Microsoft.PowerShell.Management) <https://learn.microsoft.com/vi-vn/powershell/module/microsoft.powershell.management/restart-computer?view>
197. Setting Up Python JSON Logger: A Practical Guide <https://www.dash0.com/guides/python-json-logger>
198. Customize key value in python structured (json) logging ... <https://stackoverflow.com/questions/74385061/customize-key-value-in-python-structured-json-logging-from-config-file?rq>
199. I Spent 4 Hours Debugging a 500 Error Because My Logs ... <https://medium.com/@rameshkannanyt0078/i-spent-4-hours-debugging-a-500-error-because-my-logs-were-just-strings-cc2b3e10c3cb>
200. Set Up a ServiceNow Security Incident Webhook Connection - Sumo Logic <https://www.sumologic.com/help/docs/alerts/webhook-connections/servicenow/set-up-security-incident-webhook/>
201. Event collection from Logicmonitor - ServiceNow <https://www.servicenow.com/docs/r/it-operations-management/event-management/event-collection-logicmonitor.html>
202. Supercharge Your Log Ingestion: Webhooks to SIEM Made Easy <https://www.youtube.com/watch?v=O5SaFwAMMtA>
203. Consume Webhook Events via Inngest | Setup Guide <https://www.inngest.com/docs/platform/webhooks>
204. Seeing detailed logs for webhook events <https://ohdear.app/news-and-updates/seeing-detailed-logs-for-webhook-events>

205. Webhooks <https://docs.gusto.com/app-integrations/docs/webhooks>
206. Webhook Event Log API | FusionAuth Docs <https://fusionauth.io/docs/apis/webhook-event-logs>
207. Windows Admin Center common troubleshooting steps <https://learn.microsoft.com/en-us/windows-server/manage/windows-admin-center/support/troubleshooting>
208. How to Implement Retry Logic with Exponential Backoff in ... <https://oneuptime.com/blog/post/2025-01-06-python-retry-exponential-backoff/view>
209. Structured Logging in Python - IT Guy Journals <https://www.itguyjournals.com/structured-logging-in-python/>
210. Configuring a ServiceNow Webhook - Zscaler Help Portal <https://help.zscaler.com/itdr/configuring-servicenow-webhook>
211. "Hostname mismatch" SSL error in Python - maciej skorupka <https://medium.com/@maciej.skorupka/hostname-mismatch-ssl-error-in-python-2901d465683>
212. How to solve the problem of getting HostName Verification ... https://help.salesforce.com/s/articleView?id=001118465&language=en_US&type=1
213. Why you should try pyinfra https://lobste.rs/s/rrv1hx/why_you_should_try_pyinfra
214. It is Infrastructure as Code, not Infrastructure as YAML! <https://blog.kalvad.com/infrastructure-as-yaml-is-hell/>
215. Infrastructure as code with pyinfra - cusy <https://www.cusy.io/en/blog/infrastruktur-als-code-mit-pyinfra.html>
216. pywinrm/winrm/protocol.py at master <https://github.com/diyan/pywinrm/blob/master/winrm/protocol.py>
217. ansible.windows/plugins/modules/win_reboot.py at main · ... https://github.com/ansible-collections/ansible.windows/blob/main/plugins/modules/win_reboot.py
218. win_updates get's a winrm timeout when a new network ... <https://github.com/ansible-collections/ansible.windows/issues/190>
219. The 4 Protocols Driving Security Risk in 2026: SMB, RDP, ... <https://zeronetworks.com/blog/the-4-protocols-driving-enterprise-risk-in-2026>
220. WinRM Penetration Testing <https://www.hackingarticles.in/winrm-penetration-testing/>
221. How to Test Whether WinRM Is Able to Handle ... <https://hub.ivanti.com/s/article/How-to-test-whether-WinRM-is-able-to-handle-requests>
222. How to Reboot a Remote Computer via WMI (No Extra ... <https://blog.paessler.com/howto-rebooting-a-computer-via-windows-management-instrumentation-wmi-call>

- 223. LogicMonitor Custom HTTP Integrations https://community.logicmonitor.com/forum/product-discussions-68/topic/logicmonitor-custom-http-integrations-5109/?focus_post=19505
- 224. How Webhook Events Work <https://open.manus.ai/docs/v2/webhooks-overview>
- 225. Configure Webhooks service - OpenText Documentation Portal <https://docs.microfocus.com/doc/269/24.3/configwebhook>
- 226. JSON Schema for Published REST Operation <https://docs.mendix.com/refguide/published-rest-service-json-schema/>
- 227. Integrazioni & Nodi - FlowForge <https://flowforge.automazionezeli.com/integrazioni>
- 228. Windows Server 2022 vs Windows Server 2025 <https://www.vpsbg.eu/blog/windows-server-2022-vs-windows-server-2025>
- 229. UserID Monitored server (WinRM-HTTP) gets Kerberos error. <https://live.paloaltonetworks.com/t5/general-topics/userid-monitored-server-winrm-http-gets-kerberos-error/td-p/507952>
- 230. WinRM Connectivity Issue on Windows Server Due to ... <https://www.ibm.com/support/pages/winrm-connectivity-issue-windows-server-due-conflicting-http-spns>